

Fahrer-App Android

Architektur-Explosionszeichnung

Funk Taxi Heringsdorf

de.taxiheringsdorf.app

Version 6.63.546 · 29.06.2026

Android 8+ · minSdk 22 · targetSdk 34

Alle Komponenten der nativen Android-Fahrer-App — Funktionen, Zuständigkeiten, Abhängigkeiten

Die Fahrer-App ist eine **hybride Android-App** auf Basis von **Capacitor (Ionic)**. Kern ist eine WebView die `index.html` (die Taxi-Web-App) rendert. Drumherum gibt es native Android-Komponenten für GPS-Tracking, Push-Benachrichtigungen, Anruf-Aufnahme, Email-Versand, Kartenwahl und Admin-Disposition — alles was eine reine WebApp nicht kann.

Legende: Fahrer nur Fahrer sichtbar Admin nur Admin/Patrick Beide Fahrer + Admin System läuft unsichtbar im Hintergrund

Activities — Bildschirme & Dialoge

MainActivity	<i>Capacitor BridgeActivity</i>	Beide Einstiegspunkt der App. Startet den Capacitor-WebView und lädt <code>index.html</code> . Registriert alle Capacitor-Plugins. Prüft Berechtigungen beim ersten Start. Leitet direkt weiter zu LoginActivity wenn kein User eingeloggt ist. → Startet: LoginActivity, DriverDashboardActivity, AdminDashboardActivity
LoginActivity	<i>AppCompatActivity</i>	Beide Login-Bildschirm. Firebase-Auth mit Email+Passwort und Google Sign-In. Liest nach Login die Rolle aus Firebase (<code>/users/{uid}/role</code>): <i>driver</i> → DriverDashboard, <i>admin</i> → AdminDashboard. → Startet: DriverDashboardActivity oder AdminDashboardActivity
DriverDashboard Activity	<i>AppCompatActivity</i>	Fahrer Haupt-Bildschirm des Fahrers. Zeigt aktive Fahrt, Schicht-Status, GPS-Qualität, ETA. Schicht-Start/Stop startet/stoppt ShiftForegroundService. ANNEHMEN-Button für zugewiesene Fahrten. Navigation per Google Maps. Hamburger-Menü für Stats, CRM, Dispo-Live, Aufnahmen. 15s-etaTick überwacht Service-Gesundheit und GPS-Intervall. → Startet: LoginActivity, VehiclePickerActivity, CallLogActivity, CrmSearchActivity, DispoActivity, AdminDashboardActivity, StatsActivity, CallRecordingsActivity, MapPickerActivity
AdminDashboard Activity	<i>AppCompatActivity</i>	Admin Dispositions-Zentrale für Patrick. Zeigt alle offenen Web-Anfragen (<code>/anfragen</code>) als RecyclerView. "Übernehmen"-Button konvertiert Anfrage in Ride, öffnet dann Fahrt-Bearbeiten-Dialog. Edit-Dialog hat EMAIL-BESTÄTIGUNG-Button → EmailPreviewActivity. System-Check-Badge. Neue Buchung, Anrufliste, Schicht-Editor, Fahrzeugverwaltung. → Startet: EmailPreviewActivity, CrmSearchActivity, CallLogActivity, ShiftEditorActivity, VehiclePickerActivity, MapPickerActivity, CallRecordingsActivity, LoginActivity
RideAlertActivity	<i>AppCompatActivity</i>	Fahrer Vollbild-Auftrag-Alarm — erscheint über dem Sperrbildschirm wenn FCM-Push eintrifft. Zeigt Kunde, Pickup, Ziel, Preis. Zwei Buttons: ANNEHMEN (grün) / ABLEHNEN (rot). ANNEHMEN sendet Broadcast an RideActionReceiver + öffnet DriverDashboardActivity. → Startet: DriverDashboardActivity · Sendet: RideActionReceiver-Broadcast

EmailPreview Activity	<i>AppCompatActivity</i>	Admin Email-Vorschau & Versand. Zeigt An-Adresse, Betreff, vollständigen Email-Body mit allen Fahrt-Details. Checkboxes für Stripe-Zahlungslink und Tracking-Link. SENDEN-Button schickt via Android-Email-App-Chooser. Keine direkte SMTP-Verbindung. ← Gestartet von: <i>AdminDashboardActivity, DriverDashboardActivity</i>
CrmSearchActivity	<i>AppCompatActivity</i>	Beide CRM-Kundensuche. Durchsucht /customers in Firebase nach Name/Telefon. Zeigt Kundenkarte mit letzten Fahrten, Stammrouten, Kontaktdaten. Auswahl → neue Buchung mit vorausgefüllten Kundendaten.
CallLogActivity	<i>AppCompatActivity</i>	Beide Anrufliste. Liest /callHistory aus Firebase, zeigt eingehende/ausgehende Anrufe mit Nummer, Zeitstempel, CRM-Zuordnung. Tippen auf Eintrag → CRM-Suche nach dieser Nummer.
CallRecordings Activity	<i>AppCompatActivity</i>	Beide Aufnahmen-Browser. Listet alle M4A-Dateien aus /sdcard/FunktaxiCalls/. Abspielen, Löschen, Teilen. Zeigt Anrufer-Nummer aus Dateiname.
DispoActivity	<i>AppCompatActivity</i>	Beide Live-Dispositions-Ansicht. Read-Only-Übersicht aller aktiven Fahrten und Fahrzeugpositionen. Für Fahrer: "Was ist gerade los?" ohne vollen Admin-Zugriff.
ShiftEditorActivity	<i>AppCompatActivity</i>	Admin Wochenplan-Editor. Pro Fahrzeug Dienst-Zeiten für Mo–So einstellen. "Alle gleich"-Toggle, Ausnahme-Tage. Schreibt nach /vehicleShifts/{vehicleId}. Basis für Vorbestellungs-Validierung und Auto-Zuweisung.
VehiclePickerActivity	<i>AppCompatActivity</i>	Fahrer Fahrzeug-Auswahl beim Schichtbeginn. Zeigt verfügbare Fahrzeuge aus Firebase. Auswahl setzt currentVehicleId und startet ShiftForegroundService.
MapPickerActivity	<i>AppCompatActivity</i>	Admin Adress-Picker auf Karte. Leaflet-WebView mit Nominatim-Geocoding. Tippen auf Karte → Adresse auflösen → zurück an aufrufende Activity. Verwendet für Pickup/Ziel-Auswahl in Fahrt-Bearbeiten.
DriverMapActivity	<i>AppCompatActivity</i>	Fahrer Flottenübersicht auf Karte. Zeigt alle Fahrzeug-GPS-Positionen live aus Firebase. Für Fahrer die einen Überblick über die Flotte wollen.
StatsActivity	<i>AppCompatActivity</i>	Beide Statistik-Übersicht. Zeigt Umsatz, Fahrten-Anzahl, Durchschnitts-Preis pro Zeitraum (heute/Woche/Monat). Liest aus /rides und /vehicles.
Onboarding HealthcheckActivity	<i>AppCompatActivity</i>	Fahrer Erst-Einrichtungs-Assistent. Prüft alle nötigen Berechtigungen: Standort (immer), Benachrichtigungen, Mikrofon (Aufnahme), Telefon-Status, Akku-Optimierung. Führt Fahrer Schritt für Schritt durch die Aktivierung.
MultiDayCopySheet	<i>BottomSheetDialog Fragment</i>	Admin Fahrt auf mehrere Tage kopieren. Kalender-Auswahl + Zieldaten. Erstellt mehrere Rides aus einer Vorlage (z.B. Dialyse-Serien).

Services — Hintergrunddienste

ShiftForeground Service	<i>Foreground Service</i>	System Kern des GPS-Trackings. Läuft solange die Schicht aktiv ist. FusedLocationProviderClient mit 15s Intervall. Schreibt Position via Cloud-Function-Heartbeat nach Firebase /vehicles/{id}. Dynamisches Intervall: on_way/picked_up/arrived/accepted → 15s, assigned → 30s, standby → 60s. PARTIAL_WAKE_LOCK gegen Prozess-Kill. Wird
--------------------------------	---------------------------	--

		bei App-Kill durch etaTick-Watchdog (alle 15s) automatisch neu gestartet. ← Gesteuert von: <i>DriverDashboardActivity, VehiclePickerActivity</i>
TaxiFCMService	<i>Firebase MessagingService</i>	System Push-Notifications empfangen. Firebase Cloud Messaging für neue Aufträge, Status-Updates und Admin-Nachrichten — auch bei geschlossener App / gesperrtem Bildschirm. Neuer Auftrag → startet RideAlertActivity über Sperrbildschirm + AlertSoundService. FCM-Token wird bei Schichtstart in Firebase gespeichert.
AlertSoundService	<i>Service</i>	System Alarm-Ton bei neuem Auftrag. Spielt lauten Klingelton ab wenn ein neuer Auftrag eintrifft — auch wenn das Handy auf lautlos gestellt ist. Stoppt wenn Auftrag angenommen oder abgelehnt wird.
CallRecorderService	<i>Foreground Service</i>	System Automatische Anrufaufnahme. Hört auf PhoneStateReceiver. OFFHOOK → MediaRecorder startet. IDLE → Aufnahme stoppt + speichert nach /sdcard/FunktaxiCalls/{Jahr}/{Monat}/{Tag}/+{Nummer}-{Richtung}-{ts}.m4a. ACR-Ersatz ohne Cloud-Abhängigkeit.



BroadcastReceivers — Ereignis-Empfänger

RideActionReceiver	<i>Broadcast Receiver</i>	Fahrer Annehmen/Ablehnen direkt aus der Benachrichtigung — ohne App zu öffnen. Empfängt Intents von Notification-Action-Buttons und von RideAlertActivity. HTTP-Call an Cloud-Function rideAction → Status in Firebase setzen. Öffnet danach DriverDashboardActivity mit openedFromAccept=true.
PhoneStateReceiver	<i>Broadcast Receiver</i>	System Anruf-Status überwachen. Empfängt PHONE_STATE-Broadcasts vom Android-System. OFFHOOK (Gespräch beginnt) → CallRecorderService starten. IDLE (Gespräch endet) → Service stoppen.

Capacitor-Plugins — JavaScript ↔ Native Brücke

AppUpdatePlugin	@CapacitorPlugin	System APK-Update-Prüfung. Ruft umwelt-taxi-insel-usedom.de/app/latest.json auf Strato ab. Vergleicht versionName mit installierter Version. Zeigt Update-Banner in der WebApp wenn neue Version verfügbar. Öffnet APK-Download-Link im Browser (nicht im In-App-Browser). <i>JS-Aufruf: AppUpdate.checkForUpdate()</i>
ShiftForeground Plugin	@CapacitorPlugin	Fahrer GPS-Service aus JavaScript steuern. Ermöglicht index.html den nativen ShiftForegroundService zu starten/stoppen. Gibt GPS-Koordinaten und Schicht-Status zurück. <i>JS-Aufruf: ShiftForeground.startShift({vehicleId})</i>
FCMPlugin	@CapacitorPlugin	Fahrer Push-Token an Firebase übermitteln. Gibt den aktuellen FCM-Token an die WebApp weiter damit er in /users/{uid}/fcmToken gespeichert werden kann. <i>JS-Aufruf: FCM.getToken()</i>
SmsSenderPlugin	@CapacitorPlugin	Admin SMS direkt aus JavaScript senden. Umgeht Web-SMS-Beschränkungen. Nutzt Android SmsManager für programmgesteuerten SMS-Versand (Buchungsbestätigungen, Status-Updates an Kunden). <i>JS-Aufruf: SmsSender.send({number, message})</i>
BatteryOptimization Plugin	@CapacitorPlugin	Fahrer Akku-Optimierung deaktivieren. Öffnet Android-Systemeinstellungen damit Fahrer die App von Akku-Optimierung ausnehmen kann. Wichtig damit ShiftForegroundService nachts nicht vom System abgewürgt wird. <i>JS-Aufruf: BatteryOptimization.requestIgnore()</i>
CrashlyticsPlugin	@CapacitorPlugin	System Firebase Crashlytics aus JavaScript. Ermöglicht der WebApp Fehler-Logs und Custom-Events an Firebase Crashlytics zu senden. Ergänzt native Crash-Reports um JavaScript-Fehler. <i>JS-Aufruf: CrashLytics.recordException({message})</i>

Externe Dienste — Abhängigkeiten

Firebase Realtime Database	Google Firebase	Zentraler Datenspeicher. Echtzeit-Synchronisation aller Fahrten, Fahrzeug-GPS, Benutzer, CRM-Kunden, Einstellungen. Wird von Native-App direkt (ohne Umweg über WebApp) gelesen und geschrieben. <i>Pfade: /rides · /vehicles · /users · /customers · /anfragen · /vehicleShifts</i>
Firebase Auth	Google Firebase	Authentifizierung. Email+Passwort und Google Sign-In. JWT-Token wird für alle Firebase-Zugriffe verwendet. Rolle (driver/admin) wird nach Login aus RTDB gelesen.
Firebase Cloud Messaging (FCM)	Google Firebase	Push-Notifications. Cloud Functions senden FCM-Nachrichten bei neuen Fahrten, Status-Updates, Admin-Meldungen. Funktioniert auch bei geschlossener App / gesperrtem Bildschirm.
Firebase Crashlytics	Google Firebase	Crash-Diagnose. Automatische Crash-Reports bei nativen Abstürzen. Stacktrace, Gerät, Android-Version werden in Firebase Console angezeigt. Ergänzt durch JS-Fehler via CrashlyticsPlugin.

Google Places API	<i>Google Maps Platform</i>	Adress-Autocomplete. Places SDK 4.4.1 (Places API New) für Autocomplete-Felder in Vorbestellungs- und CRM-Adress-Eingaben. Legacy Places API wurde Anfang 2025 abgeschaltet.
Firebase Cloud Functions	<i>Google Firebase</i>	Server-Logik. rideAction-Endpoint für Annehmen/Ablehnen, Heartbeat-Empfang mit GPS-Update, Telegram-Webhook, Auto-Zuweisung, Konflikt-Auflösung. Native App kommuniziert direkt per HTTPS mit den Functions.
Strato-Server	<i>umwelt-taxi-insel-usedom.de</i>	APK-Hosting. Aktuellste APK liegt unter /app/taxi-app-latest.apk, Versions-Info unter /app/latest.json. GitHub Actions deployen bei jedem Merge automatisch. AppUpdatePlugin prüft diese URL.

Datenfluss — Neue Fahrt von Anfrage bis GPS-Tracking

Schritt	Wohin	Was passiert
1. Kunde bucht <i>Telegram / Web / WhatsApp</i>	→ Firebase	Cloud Function legt neue Anfrage in /anfragen/{id} an. System-Check wird aktualisiert.
2. Admin sieht Anfrage <i>AdminDashboardActivity</i>	← Firebase	Firebase-Listener empfängt neue Anfrage, RecyclerView zeigt sie mit Kundendaten und gewünschter Uhrzeit.
3. Übernehmen <i>AdminDashboardActivity</i>	→ Firebase	Cloud Function erstellt Ride in /rides/{id}, löscht Anfrage. Danach öffnet sich Fahrt-Bearbeiten-Dialog automatisch.
4. Email-Bestätigung <i>EmailPreviewActivity</i>	→ Email-App	EmailPreviewActivity öffnet sich mit allen Fahrt-Daten. Patrick prüft, tippt SENDEN → Android-Email-App-Chooser öffnet sich.
5. Auto-Zuweisung <i>Cloud Function (alle 10 Min)</i>	→ Firebase	scheduledAutoAssign wählt passendes Fahrzeug laut Schichtplan. Schreibt assignedVehicle in Ride.
6. FCM-Push an Fahrer <i>TaxiFCMService</i>	← FCM	TaxiFCMService empfängt Push-Nachricht. Startet RideAlertActivity über Sperrbildschirm + AlertSoundService (Alarm-Ton).
7. Fahrer nimmt an <i>RideAlertActivity / RideActionReceiver</i>	→ Cloud Fn	Broadcast an RideActionReceiver → HTTPS-Call an rideAction-Function → Status = "accepted" in Firebase.
8. GPS-Tracking läuft <i>ShiftForegroundService</i>	→ Firebase	Bei Status "accepted" wechselt Heartbeat auf 15s. GPS-Position alle 15s via Cloud Function → /vehicles/{id}. Admin sieht Fahrer live auf der Karte.

Build & Deployment

Was	Trigger	Ergebnis
APK-Build	Push auf main (wenn android/** geändert)	GitHub Actions → signierte Release-APK → automatisch auf Strato hochgeladen. Fahrer-App zeigt Update-Banner beim nächsten Öffnen.
Version hochzählen	Pflicht bei jeder Android-Änderung	android/app/build.gradle: versionCode + versionName erhöhen. Format: Major×1.000.000 + Minor×1.000 + Patch
Signing	GitHub Actions Secrets	ANDROID_KEYSTORE_PATH, ANDROID_KEYSTORE_PASSWORD, ANDROID_KEY_ALIAS, ANDROID_KEY_PASSWORD — lokal nicht vorhanden.

